



Build and Battle Your Trading Agent

HACKATHON

28 May 2026

Agenda

- 12:30–13:30 Set the Scene & Kickoff
- 13:30–15:30 Build Your Agent
- 15:30–16:00 Coffee Break
- 16:00–17:00 Prepare Materials
- 17:00–18:00 Show and Tell
- 18:00–19:00 Agent Tournament & Wrap-up

Deliverables for Today

- ▷ The code you produced during the hackathon
- ▷ A short presentation (cover + 2 slides) describing your agent's design and strategy. Each team has 5 minutes to present during Show and Tell, so be clear and concise!

Deliverables for 7 June

- ▷ Each team will submit a written report (around 1000 words plus figures), containing a brief description of the ABIDES framework and a discussion of your agent's design and results

ABIDES

ABIDES — Agent-Based Interactive Discrete Event Simulation

Provides a realistic, configurable environment in which a population of heterogeneous **agents** interact through a simulated exchange, reproducing the microstructural dynamics of equity markets

ABIDES is designed for research in algorithmic trading and market microstructure, and for training and evaluating RL-based trading agents without risking real capital

Byrd, D. *et al.* (2020). *ABIDES: Towards high-fidelity market simulation for AI research*. ACM ICAIF.

The **kernel** is the central engine of the simulation. It implements **discrete-event simulation** (DES): time advances from one event to the next rather than in fixed steps, making the simulator efficient even over long time horizons

The kernel maintains a priority queue of events of the form $(t, \text{agent}_i, \text{message})$ and dispatches each message to the target agent at the scheduled simulation time t

Agents interact with the kernel to:

- ▷ schedule their own future wakeups
- ▷ send messages to other agents (with optional latency)
- ▷ query the current simulation time

Every participant in the simulation is an **agent**: a self-contained object with a unique ID, private state, and a set of message handlers

Agents can only communicate via the kernel's message passing mechanism; they have no direct access to each other's state, mimicking the information asymmetry of real markets

ExchangeAgent	Runs the limit order book
NoiseAgent	Trades randomly (uninformed)
ValueAgent	Tracks a mean-reverting fundamental value
MomentumAgent	Follows recent price trends
MarketMakerAgent	Continuously quotes bid and ask

All **inter-agent communication** is asynchronous and routed through the kernel. Each message carries a scheduled delivery time, allowing the simulation to model realistic network and processing latencies

Key message types:

- ▷ **Wakeup**: the kernel calls an agent at a self-scheduled time, enabling agents to implement periodic strategies without external triggers
- ▷ **LimitOrderMsg / MarketOrderMsg**: a trading agent instructs the exchange to place an order on its behalf
- ▷ **CancelOrderMsg**: request to withdraw an outstanding limit order
- ▷ **OrderExecutedMsg**: the exchange notifies an agent that its order (or part of it) has been filled
- ▷ **MarketDataMsg**: end-of-period LOB snapshot broadcast to subscribed agents

The **ExchangeAgent** is the only agent that maintains the limit order book (LOB). It receives order messages from trading agents, applies price-time priority matching, and sends execution confirmations

Upon receiving a new order the exchange:

1. checks available liquidity on the opposite side of the book
2. fills as much as possible at current prices, generating **OrderExecutedMsg** notifications
3. adds any unfilled remainder as a resting limit order
4. optionally broadcasts an updated LOB snapshot to subscribers

By configuring the population of agents, researchers can reproduce stylized microstructural facts and stress-test algorithmic strategies in a controlled, reproducible environment

The **TradingAgent** class is the base class from which all market participants (except the exchange) inherit. It hides the message-passing boilerplate and exposes a small API to place, cancel and track orders, plus private state for holdings, cash and the latest known bid/ask

When subclassing **TradingAgent**, the minimum set of methods to override:

- ▷ **__init__**: store strategy parameters and call `super().__init__(...)` to register id, name and type
- ▷ **kernelStarting(startTime)**: call `super()`, then subscribe to market data and request a first wakeup
- ▷ **wakeup(currentTime)**: the decision loop — read the agent's view of the book, decide what to do, send order messages, and schedule the next wakeup
- ▷ **receiveMessage(currentTime, msg)**: dispatch on the message type (`OrderExecuted`, `OrderAccepted`, `MarketData`, ...) to update state and react; call `super()` first so accounting stays correct
- ▷ **kernelStopping()**: finalize bookkeeping, compute MtM and P&L

RMSC04 (Reference Market Simulation Configuration, v4) is the canonical ABIDES setup designed to reproduce a realistic trading day for a liquid US equity

A single Python config file wires together the kernel, the exchange, and a calibrated population of background agents on top of a mean-reverting fundamental price series

Default agent population on ticker ABM:

- 1 ExchangeAgent (limit order book)
- 2 Adaptive POV MarketMakerAgents
- 102 ValueAgents
- 12 MomentumAgents
- 1000 NoiseAgents
- (+1) optional POV ExecutionAgent (your strategy)

The mix is tuned so that mid-price tracks the fundamental and the book exhibits realistic spread, depth and volume profiles

Key ingredients of the RMSC04 config:

- ▷ **Fundamental:** sparse discrete mean-reverting (OU-like) process; value agents observe it with Gaussian noise and update their belief in a Bayesian way
- ▷ **Noise agents:** wake up once each, trade a fixed size in a random direction; arrival times follow a U-shape, denser near open and close
- ▷ **Momentum agents:** compare a 20-step and a 50-step moving average of the mid-price and trade in the direction of the trend
- ▷ **POV market makers:** skip a window around the mid-price and post a ladder of quotes outside it; the order size is a percentage of recently transacted volume, the window width is an EWMA of the observed spread
- ▷ **Latencies:** per-agent log-normal network delays and computation delays, sampled by the kernel for every message

Build Your Agent Challenge

Three Alternative Strategy Briefs

Trend follower

When price breaks out of a recent trading range it tends to keep moving in that direction. Implement a breakout strategy that tries to capture these trends.

Mean-reversion trader

Prices tend to revert to their recent average. When the current mid-price deviates significantly from its rolling mean, trade against the deviation and close the position once the price normalises.

Liquidity provider

Earn the bid-ask spread by posting a limit buy slightly below and a limit sell slightly above the current mid-price, then refreshing quotes periodically. Manage inventory risk by skewing quotes. Cancel outstanding orders before reposting.

